```
    key.i = 512 ;
    printf ( "\nkey.i = %d", key.i ) ;
    printf ( "\nkey.ch[0] = %d", key.ch[0] ) ;
    printf ( "\nkey.ch[1] = %d", key.ch[1] ) ;

    key.ch[0] = 50 ; /* assign a new value to key.ch[0] */
    printf ( "\nkey.i = %d", key.i ) ;
    printf ( "\nkey.ch[0] = %d", key.ch[0] ) ;
    printf ( "\nkey.ch[1] = %d", key.ch[1] ) ;
}
```

And here is the output...

```
key.i = 512
key.ch[0] = 0
key.ch[1] = 2
key.i= 562
key.ch[0] = 50
key.ch[1] = 2
```

Before we move on to the next section, let us reiterate that a union provides a way to look at the same data in several different ways. For example, there can exist a union as shown below.

```
union b
{
    double d ;
    float f[2] ;
    int i[4] ;
    char ch[8] ;
} ;
union b data ;
```

In what different ways can the data be accessed from it? Sometimes, as a complete set of eight bytes (**data.d**), sometimes as two sets of 4 bytes each (**data.f[0]** and **data.f[1]**), sometimes as

four sets of 2 bytes each (**data.i[0]**, **data.i[1]**, **data.i[2]** and **data.[3]**) and sometimes as eight individual bytes (**data.ch[0]**, **data.ch[1]... data.ch[7]**).

Also note that, there can exist a union, where each of its elements is of different size. In such a case, the size of the union variable will be equal to the size of the longest element in the union.

Union of Structures

Just as one structure can be nested within another, a union too can be nested in another union. Not only that, there can be a union in a structure, or a structure in a union. Here is an example of stuctures nested in a union.

```
# include <stdio.h>
void main( )
{
    struct a
    {
        int i ;
        char c[2] ;
    } ;
    struct b
    {
        int j ;
        char d[2] ;
    } ;
    union z
    {
        struct a key ;
        struct b data ;
    } ;
    union z strange ;

    strange.key.i = 512 ;
    strange.data.d[0] = 0 ;
```